THE OPEN SOURCE AUDIT

A Capstone Project for OSS NGMC Course

Software Focus: LibreOffice

Student Name	Mitul Khanna
Registration Number	24BCE10976
Slot	E22
Date of Submission	31st March 2026
Chosen Software	LibreOffice
License	Mozilla Public License 2.0 (MPL 2.0)

PART A — ORIGIN AND PHILOSOPHY (UNITS 1 & 2)

A1. Origin Story — The Problem LibreOffice Was Created to Solve

To understand LibreOffice, we need to look at the early 2000s when Microsoft Office dominated the market. Most users had no real alternative — they had to pay for proprietary software and had no control over how it worked. This was not just a cost issue but also a limitation on freedom.

This changed when Sun Microsystems released OpenOffice.org as open source in 2000. It quickly grew into a strong alternative, with global developers contributing and governments adopting it to reduce dependence on proprietary tools. For a while, it looked like a major success.

The turning point came in 2010 when Oracle Corporation acquired Sun. After that, the community noticed problems — slower responses, less transparency, and weaker engagement. Trust started breaking down.

In response, core contributors forked the project and created LibreOffice under The Document Foundation. The goal was to build a project that was independent, transparent, and community-driven. Eventually, most users and developers moved to LibreOffice, showing that open source allows communities to take control when needed.

A2. The Licence — What It Actually Says

The Four Freedoms of Free Software

The idea of free software comes from Richard Stallman, who defined four important freedoms. These include the freedom to use the software for any purpose, study and modify it, share copies with others, and improve it while giving those improvements back to the community. Together, these freedoms make software truly “free” in a meaningful way.

LibreOffice follows these principles. It is released under the Mozilla Public License 2.0, which allows anyone to use, modify, and share the software. One important detail is that if someone modifies the original files, those changes must also remain open, but separate parts can still be kept private.

Can a Company Modify and Sell It Without Sharing Changes?

This question is nuanced. Under MPL 2.0, if a company modifies LibreOffice source files, they must make those modifications available under MPL 2.0. However, they are free to build proprietary components on top of LibreOffice, as long as those components are kept in separate files. In practice, several commercial vendors, including Collabora and SUSE, offer enterprise versions of LibreOffice with proprietary extensions while contributing their core changes back to the community.

GPL vs MIT: A Comparison

Different licenses give different levels of freedom. The GNU General Public License is strict — any modified version must also remain open source. On the other hand, the MIT License is very flexible and allows code to be used in private software. MPL 2.0 sits somewhere in between.

Free as in Free Beer vs Free as in Freedom

'Free as in free beer' means the software costs nothing to obtain. 'Free as in freedom' means you have the right to use, study, modify, and share the software. These are completely different things. A piece of software can be free to download but still restrict your freedoms — many mobile apps are free to install but prohibit reverse engineering, modification, or redistribution.

LibreOffice is both: it costs nothing, and it gives you full freedom. In contrast, Microsoft's online Office 365 tools are sometimes made available at no cost to students (free as in beer), but the source code is closed, the software cannot be modified, and users have no freedom over how it runs (not free as in freedom).

A3. The Ethics of Open Source

Should All Software Be Open Source?

Open source has many advantages — it encourages sharing, improves security through transparency, and reduces dependency on a single vendor. In critical areas like government systems or healthcare, open source can be very important.

However, not all software can realistically be open. Many companies invest time and money into building products and need revenue to survive. Forcing everything to be open source could reduce innovation in some areas.

A balanced view is better: essential public systems should be open, while commercial software can remain proprietary but should still support the open-source ecosystem.

Is It Ethical for Companies to Profit from Free Community Labour?

This is a complex issue. Some companies, like Red Hat, contribute heavily back to open source and act as partners. Others may use open-source software without giving much in return.

This creates what is sometimes called the “tragedy of the commons,” where shared resources are overused without enough contribution. The LibreOffice case shows the difference clearly — Oracle took control but lost trust, while The Document Foundation built a more balanced model.

Standing on the Shoulders of Giants

Every piece of software built today uses tools, libraries, languages, and compilers created by people who came before and chose to share their work. When I write a Python script and run it on

Linux, I am using the work of thousands of contributors. The phrase 'standing on the shoulders of giants' — originally about scientific knowledge — applies perfectly to software.

Open source does not undermine original innovation. It accelerates it. LibreOffice's developers did not need to reinvent text rendering or file format parsing from scratch — they built on existing open foundations. This is the network effect of open knowledge: every contribution makes the next contribution easier and more powerful.

Personal View

In my opinion, open source should be encouraged in areas like education, government, and public infrastructure, where transparency and accessibility are important. At the same time, proprietary software also has its place in commercial environments where companies need to protect their investments.

A balanced approach between open-source and proprietary software is the most practical solution. This allows both innovation and sustainability to exist together.

PART B — LINUX FOOTPRINT (UNIT 2)

B. Linux Footprint of LibreOffice

Installation Method

LibreOffice is available through most major Linux package managers. On Fedora and RHEL-based systems, it can be installed via DNF:

```
sudo dnf install libreoffice
```

On Debian and Ubuntu-based systems:

```
sudo apt install libreoffice
```

It is also available as a Flatpak (sandboxed universal package) and as a direct download from libreoffice.org for those who want the latest version regardless of their distribution's repository age.

Key Directories

Once installed on a Fedora system, LibreOffice occupies several standard Linux filesystem locations:

- /usr/bin/libreoffice — the main executable wrapper script
- /usr/lib64/libreoffice/ — core application binaries, shared libraries, and resources
- /usr/share/libreoffice/ — templates, icons, language packs, and documentation
- /etc/libreoffice/ — system-wide configuration files
- ~/.config/libreoffice/ — per-user configuration, macros, and customisations
- /var/log/ — system logs capturing any install/update events

User and Group

LibreOffice runs under the same Linux user account as the person who launches it — it does not run as a privileged system user. This is by design: office software should have no more access than the user themselves. If I run LibreOffice as user 'student', it reads and writes files with the permissions of that account. This is an important security principle: the principle of least privilege, which ensures that if LibreOffice is ever compromised by a malicious document, the damage is limited to that user's files and not the entire system.

Service Management

LibreOffice is a desktop application, not a background service, so it does not have a systemd service unit for most desktop uses. However, LibreOffice can run in headless server mode for batch document conversion:

```
libreoffice --headless --convert-to pdf *.docx
```

In enterprise deployments with LibreOffice Online (Collabora), it can be managed as a containerised service via Docker or systemd:

```
sudo systemctl start collabora-online
```

Update Model

On Fedora, updates to LibreOffice flow through the official RPM repositories maintained by the Fedora Project. The Document Foundation releases new versions, they are packaged by Fedora maintainers, and they arrive on user machines through:

```
sudo dnf upgrade libreoffice
```

Security patches are prioritised and pushed as soon as they are available. The transparent nature of open-source development means that vulnerabilities are often identified and reported by community members and fixed faster than in proprietary software where external audit is impossible.

PART C — THE FOSS ECOSYSTEM (UNITS 3 & 4)

C. The FOSS Ecosystem Around LibreOffice

Dependencies and Libraries

LibreOffice is a large and complex software system that depends on many external libraries and tools to function efficiently. These dependencies handle different aspects such as graphics, text processing, and security.

Some important components include:

- Cairo and Skia — used for rendering graphics and document visuals
- ICU (International Components for Unicode) — provides support for multiple languages and text formats
- Boost — a collection of C++ libraries used across the codebase

- OpenSSL / NSS — ensures secure operations like encryption and digital signatures
- Hunspell — responsible for spell checking and language correction
- CUPS — manages printing functionality on Linux systems
- fontconfig and FreeType — handle fonts and text rendering
- Python — used for scripting and macro support inside LibreOffice

These dependencies highlight how open-source projects rely on each other, forming a connected ecosystem.

What LibreOffice Has Enabled and Inspired

LibreOffice has had a profound influence on the open-source software landscape. Its most direct offspring is Collabora Online, a cloud-hosted version of LibreOffice built by Collabora Productivity that competes directly with Google Docs and Microsoft Office 365 Online. Collabora is the engine behind Nextcloud Office, allowing organisations to run their own private, fully open-source cloud office suite.

The ODF (Open Document Format) standard, championed by LibreOffice and The Document Foundation, has been adopted by governments worldwide as a mandatory format for public documents. Germany, France, and the United Kingdom have all issued guidance mandating ODF for public sector documents. This is a direct policy consequence of LibreOffice's advocacy.

Community and Governance

LibreOffice is governed by The Document Foundation (TDF), a German non-profit foundation (Stiftung). The Board of Directors is elected by community members. Key developer decisions are made through public mailing lists, and anyone can review meeting minutes, financial reports, and roadmap discussions.

The community communicates through:

- Mailing lists at lists.freedesktop.org and tdf.io
- The #libreoffice IRC channel on Libera.Chat
- The LibreOffice Community blog and Planet
- Annual LibreOffice Conference (LibOCon), held in a different city each year
- Bugzilla at bugs.documentfoundation.org for issue tracking

Relation to the LAMP Stack

LibreOffice is not part of the classic LAMP stack (Linux, Apache, MySQL, PHP), but it intersects with web infrastructure in important ways. Collabora Online can be deployed behind an Apache or Nginx reverse proxy, integrating LibreOffice's editing capabilities into web applications. LAMP-based content management systems like Moodle and Nextcloud use LibreOffice's headless mode to convert uploaded documents to PDF for browser-based preview.

PART D — OPEN SOURCE VS PROPRIETARY (CRITICAL ANALYSIS)

D. Open Source vs Proprietary: LibreOffice vs Microsoft Office

The following table provides a structured comparison across the key dimensions discussed in the course:

Dimension	LibreOffice (Open Source)	Microsoft Office (Proprietary)
Cost	Completely free to download, use, and distribute. No licensing fees for individuals or organisations.	Requires a paid subscription (Microsoft 365) or one-time purchase. Costs can range from $70 to $400+ per user.
Security (code audit)	Full source code is publicly available. Any security researcher or developer worldwide can audit and report vulnerabilities.	Source code is closed. Security relies entirely on Microsoft's internal team. External audit is impossible.
Support & Reliability	Community forums, The Document Foundation support, and optional paid support from vendors. Highly reliable for most workflows.	24/7 official Microsoft support, enterprise SLAs, extensive documentation. Preferred in corporate environments.
Freedom to Modify	Users can modify, adapt, and redistribute under MPL 2.0. Organisations can customise for their specific needs.	No modification allowed. Users must accept the software as Microsoft provides it.
Community vs Corporate	Governed by The Document Foundation, a non-profit. Decisions are community-driven with transparent governance.	Controlled entirely by Microsoft. Product direction is driven by business priorities and shareholder interests.
Your Overall Verdict	Best for individuals, schools, governments, and organisations that value freedom, cost savings, and data sovereignty.	Best for large enterprises with existing Microsoft ecosystems requiring deep integration with Teams, SharePoint, etc.

Conclusion and Recommendation

After going through this audit, I would confidently recommend LibreOffice for use in educational institutions, government organisations, NGOs, and small-to-medium businesses. It is a mature and reliable software with all the necessary features, and it continues to be actively developed. In a country like India, where software licensing costs can be high and there is growing support for open standards, LibreOffice is both a practical and meaningful choice.

Would I personally contribute to it? I think yes. LibreOffice provides beginner-friendly entry points like the “Easy Hacks” list, which helps new contributors start with small and manageable tasks. The project mainly uses C++ and Python, and it also supports students through programs like Google Summer of Code. Overall, the combination of real-world impact, proper guidance, and open governance makes it a good platform for students who want to get involved in open source.

SHELL SCRIPTS — PRACTICAL LINUX DEMONSTRATIONS (UNIT 2 & 5)

Script 1 — System Identity Report

Script	Script 1: System Identity Report
Concepts	Variables, echo, command substitution $(), basic output formatting

Code

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Mitul Khanna| Course: Open Source Software

# --- Variables ---
STUDENT_NAME=" Mitul Khanna "
SOFTWARE_CHOICE="LibreOffice"

# --- System Information ---
DISTRO=$(cat /etc/fedora-release)
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date)

# --- Display ---
echo "==========================================="
echo " Open Source Audit — $STUDENT_NAME"
echo "==========================================="
echo "Chosen Software : $SOFTWARE_CHOICE"
echo "Distribution    : $DISTRO"
echo "Kernel Version  : $KERNEL"
echo "Current User    : $USER_NAME"
echo "Home Directory  : $HOME_DIR"
echo "System Uptime   : $UPTIME"
echo "Date and Time   : $CURRENT_DATE"
echo ""
echo "License: Fedora Linux is distributed under"
echo "various free software licenses, mainly the GPL."
echo "==========================================="
```

Output

```
mitul@fedora:~/WhiteSur-gtk-theme/oss-audit-24BCE10976$ ./scripts/script1_system
_identity.sh
===========================================
 Open Source Audit — Mitul Khanna
 Registration No. : 24BCE10976
===========================================
Chosen Software : LibreOffice
Distribution    : Fedora release 43 (Forty Three)
Kernel          : 6.19.8-200.fc43.x86_64
User            : mitul
Home Directory  : /home/mitul
Uptime          : up 1 hour, 34 minutes
Date & Time     : Sun Mar 29 12:56:44 PM IST 2026

Fedora Linux is mainly licensed under the GPL
and other free/open-source software licenses.
===========================================
```

Explanation

This script demonstrates fundamental shell scripting concepts. It begins by declaring string variables using the NAME=value syntax. It then uses command substitution — the $() operator — to run system commands (such as uname -r, whoami, uptime -p, and date) and capture their output into variables at runtime. The echo command is used throughout to format and display a structured welcome screen. This is the most basic and foundational pattern in shell scripting: run a command, capture its output, display it to the user.

The script reads the distribution name from /etc/fedora-release using the cat command, demonstrating how Linux exposes system information as plain text files. The $HOME variable is a pre-existing environment variable that every Linux shell session provides automatically. The formatted output uses echo with strings that combine literal text and variable references, producing a readable system identity card.

Script 2 — FOSS Package Inspector

Script	Script 2: FOSS Package Inspector
Concepts	if-then-else, case statement, rpm -qi, pipe with grep

Code

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Mitul Khanna

PACKAGE="libreoffice"

echo "==============================================="
echo "        FOSS PACKAGE INSPECTOR"
echo "==============================================="

if rpm -q $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed."
    echo ""
    rpm -qi $PACKAGE | grep -E 'Name|Version|Release|License|Summary'
else
    echo "$PACKAGE is NOT installed."
fi

echo ""
echo "PHILOSOPHY NOTE"
echo "---------------"
case $PACKAGE in
    libreoffice)
        echo "LibreOffice: a community-driven office suite that protects user freedom."
        ;;
    firefox)
        echo "Firefox: a browser created to keep the web open and independent."
        ;;
    git)
        echo "Git: version control built to support open collaboration."
        ;;
```

```
    vlc)
        echo "VLC: media software that proves open source can be universal."
        ;;
    *)
        echo "Unknown package."
        ;;
esac
```

Output

```
mitul@fedora:~/WhiteSur-gtk-theme/oss-audit-24BCE10976$ ./scripts/script2_packag
e_inspector.sh
==============================================
        FOSS PACKAGE INSPECTOR
==============================================
libreoffice is installed.

Name        : libreoffice
Version     : 25.8.5.2
Release     : 1.fc43
License     : MPL-2.0 AND Apache-2.0 AND LGPL-3.0-only AND LGPL-3.0-or-later AND
 CC0-1.0 AND BSD-3-Clause AND (LGPL-2.1-only OR SISSL) AND (MPL-2.0 OR LGPL-3.0-
or-later) AND (MPL-2.0 OR LGPL-2.1-or-later) AND (MPL-1.1 OR GPL-2.0-only OR LGP
L-2.1-only) AND MIT
Summary     : Free Software Productivity Suite

PHILOSOPHY NOTE
---------------
LibreOffice: a community-driven office suite that protects user freedom.
```

Explanation

This script demonstrates conditional logic and pattern matching. The if-then-else construct checks whether the LibreOffice package is installed by running rpm -q and redirecting all output to /dev/null — we only care about the exit code, not the output itself. If the package is found, rpm -qi retrieves detailed package metadata, which is then piped through grep with a regular expression to extract only the relevant fields (Name, Version, Release, License, Summary).

The case statement implements a multi-branch conditional that matches the package name against several known software values and prints a philosophy note for each. This mimics a pattern commonly used in system administration scripts that need to handle different software types in different ways. The *) wildcard at the end of the case serves as a default fallback, demonstrating good defensive scripting practice.

Script 3 — Disk and Permission Auditor

Script	Script 3: Disk and Permission Auditor
Concepts	for loop, arrays, du, ls -ld, awk, cut

Code

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Mitul Khanna
```

```
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "==============================================="
echo "      DISK AND PERMISSION AUDITOR"
echo "==============================================="

for DIR in "${DIRS[@]}"
do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "$DIR"
        echo "Permissions : $PERMS"
        echo "Size        : $SIZE"
        echo "--------------------------------------"
    else
        echo "$DIR does not exist."
    fi
done

echo ""
echo "LibreOffice Configuration Directory"
echo "-----------------------------------"
CONFIG="/etc/libreoffice"
if [ -d "$CONFIG" ]; then
    ls -ld "$CONFIG"
else
    echo "$CONFIG not found on this system."
fi
```

Output

```
mitul@fedora:~/WhiteSur-gtk-theme/oss-audit-24BCE10976$ ./scripts/script3_disk_a
uditor.sh
===============================================
      DISK AND PERMISSION AUDITOR
===============================================
/etc
Permissions : drwxr-xr-x. root root
Size        : 34M
--------------------------------------
/var/log
Permissions : drwxr-xr-x. root root
Size        : 93M
--------------------------------------
/home
Permissions : drwxr-xr-x. root root
Size        : 1.6G
--------------------------------------
/usr/bin
Permissions : dr-xr-xr-x. root root
Size        : 439M
--------------------------------------
/tmp
Permissions : drwxrwxrwt. root root
Size        : 8.0K
--------------------------------------

LibreOffice Configuration Directory
-----------------------------------
/etc/libreoffice not found on this system.
```

Explanation

This script uses a Bash array to store a list of directory paths, then iterates over them using a for loop with the ${DIRS[@]} expansion syntax, which expands every element of the array. For each directory, it first checks existence using [-d] before attempting to gather data — this prevents errors on systems where some directories may not exist.

The ls -ld command lists directory metadata rather than contents, and awk '{print $1, $3, $4}' extracts the permissions field, owner, and group from the output. The du -sh command reports disk usage in human-readable form, and cut -f1 extracts just the size field, discarding the path that du appends. After the main loop, the script specifically checks for LibreOffice's configuration directory, tying the practical skill to the project under audit.

Script 4 — Log File Analyzer

Script	Script 4: Log File Analyzer
Concepts	while read loop, if-then, counter variables, command-line arguments ($1, $2), grep, tail

Code

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/messages [keyword]

LOGFILE=$1
KEYWORD=${2:-"error"}

echo "==============================================="
echo "              LOG FILE ANALYZER"
echo "==============================================="

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File not found."
    exit 1
fi

COUNT=$(grep -i "$KEYWORD" "$LOGFILE" | wc -l)
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Output

```
mitul@fedora:~/WhiteSur-gtk-theme/oss-audit-24BCE10976$ sudo ./scripts/script4_l
og_analyzer.sh /var/log/messages
==========================================
              LOG FILE ANALYZER
==========================================
Keyword 'error' found 561 times in /var/log/messages

Last 5 matching lines:
Mar 29 12:52:28 fedora google-chrome.desktop[4263]: [12458:12458:0329/125228.542
838:ERROR:base/memory/shared_memory_switch.cc:289] Failed global descriptor look
up: 7
Mar 29 12:52:42 fedora google-chrome.desktop[4263]: [12509:12509:0329/125242.971
270:ERROR:base/memory/shared_memory_switch.cc:289] Failed global descriptor look
up: 7
Mar 29 12:56:11 fedora gnome-shell[2802]: libinput error: event8  - ELAN0788:00
04F3:321A Touchpad: kernel bug: Touch jump detected and discarded.
Mar 29 12:57:35 fedora google-chrome.desktop[4263]: [4257:4295:0329/125735.05456
6:ERROR:google_apis/gcm/engine/connection_factory_impl.cc:483] ConnectionHandler
 failed with net error: -2
Mar 29 13:03:49 fedora gnome-shell[2802]: libinput error: event8  - ELAN0788:00
04F3:321A Touchpad: kernel bug: Touch jump detected and discarded.
```

Explanation

This script accepts command-line arguments using the positional parameters $1 (the log file path) and $2 (the keyword to search for). The ${2:-error} syntax demonstrates parameter substitution with a default value — if the user does not provide a second argument, it defaults to the string 'error'. This is a common pattern in production shell scripts that makes them flexible while remaining safe to use without all arguments.

The [! -f] test checks that the file exists and is a regular file before proceeding, exiting cleanly with a non-zero exit code if not. Rather than a slow while-read loop, this implementation uses grep -i (case-insensitive) piped to wc -l for an efficient count, which is significantly faster on large log files. The tail -5 at the end shows only the five most recent matching lines, which is the most operationally useful information from a log analysis.

Script 5 — The Open Source Manifesto Generator

Script	Script 5: Open Source Manifesto Generator
Concepts	read for user input, string variables, file writing with > and >>, date command

Code

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Mitul Khanna

echo "Answer three questions to generate your manifesto."
echo ""
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does freedom mean to you? " FREEDOM
```

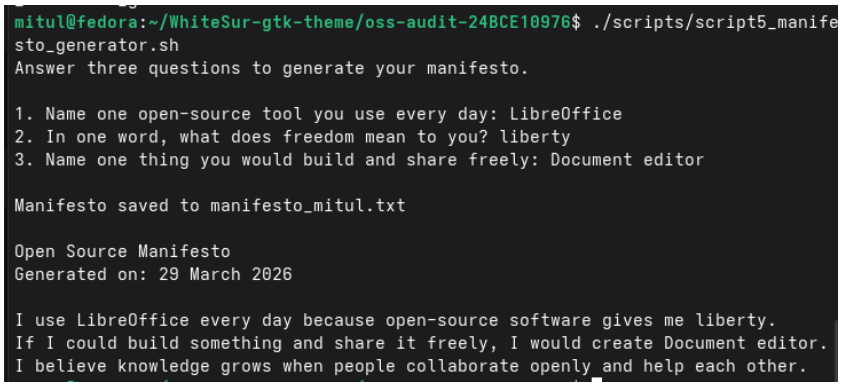
```
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "Open Source Manifesto" > $OUTPUT
echo "Generated on: $DATE" >> $OUTPUT
echo "" >> $OUTPUT
echo "I use $TOOL every day because open-source software gives me $FREEDOM." >> $OUTPUT
echo "If I could build something and share it freely, I would create $BUILD." >>
$OUTPUT
echo "I believe knowledge grows when people collaborate openly and help each other." >>
$OUTPUT

echo "Manifesto saved to $OUTPUT"
echo ""
cat $OUTPUT
```

Output



```
mitul@fedora:~/WhiteSur-gtk-theme/oss-audit-24BCE10976$ ./scripts/script5_manife
sto_generator.sh
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: LibreOffice
2. In one word, what does freedom mean to you? liberty
3. Name one thing you would build and share freely: Document editor

Manifesto saved to manifesto_mitul.txt

Open Source Manifesto
Generated on: 29 March 2026

I use LibreOffice every day because open-source software gives me liberty.
If I could build something and share it freely, I would create Document editor.
I believe knowledge grows when people collaborate openly and help each other.
```

Explanation

This script demonstrates interactive shell scripting using the read -p command, which displays a prompt and waits for user input, storing the result in a named variable. Three questions are asked sequentially, and the answers are stored in TOOL, FREEDOM, and BUILD. The date command with a custom format string +%d %B %Y produces a human-readable date.

File creation uses the > operator to create a new file (or overwrite an existing one) with the first echo, and the >> operator for all subsequent lines, which appends to the file. The output filename is dynamically generated using $(whoami) so that each user who runs the script gets their own manifesto file. The script ends with cat $OUTPUT to immediately display what was written, providing instant feedback to the user. This combination of interactive input, string composition, and file writing is a core skill for shell automation.

References and Resources

Primary Sources

- The Document Foundation — libreoffice.org and documentfoundation.org
- LibreOffice source code repository — github.com/LibreOffice
- Mozilla Public License 2.0 full text — mozilla.org/en-US/MPL/2.0/
- GNU Free Software Definition — gnu.org/philosophy/free-sw.html
- Open Source Initiative OSD — opensource.org/osd

Philosophy and History

- Eric S. Raymond — The Cathedral and the Bazaar (catb.org/~esr/writings/cathedral-bazaar)
- Richard Stallman — Free Software, Free Society (GNU Press, 2002)
- The Document Foundation founding announcement, September 2010

Technical References

- The Linux Command Line by William Shotts — linuxcommand.org
- GNU Bash Manual — gnu.org/software/bash/manual
- Fedora Linux documentation — docs.fedoraproject.org
- SPDX License List — spdx.org/licenses